

NAVFarm — How the System Works

Verified against a running instance on 2026-09-01. Every figure, screenshot and behaviour below was checked in the browser and against the database, not inferred from test output.

- API: NestJS on `http://localhost:2877/api/v1`
- Web: Next.js on `http://localhost:3002`
- Databases: MySQL — `navfarm_master` (15 tables), `tenant_system` (101 tables), `tenant_devco` (101 tables)
- Migrations: 49 applied to each tenant database, zero schema drift against `schema.ts`

1. The shape of the system

NAVFarm is a multi-tenant livestock ERP. Three ideas carry most of the design:

Database-per-tenant. `navfarm_master` holds the tenant directory and platform admins. Each tenant gets its own database (`tenant_<code>`) with the full 101-table schema. A request carries `x-tenant-id`; the CLS-scoped Drizzle connection is resolved per request, so tenant isolation is structural rather than a `WHERE tenant_id = ?` convention that one missed clause can break.

Three workspace scopes. The same console renders as three different products depending on where you are standing — the whole tenant, one company, or one operational area. This is the single most important thing to understand about the UI, and section 2 covers it.

Batches are the unit of production; animals are the unit of identity. A batch carries cost and stage; an animal register row carries a tag, a parity count and its own stage/pen/batch pointers. The two are joined, not merged — which is what makes "move some of the animals but not the batch" expressible at all (section 5).

The demo tenant

`devco` — Dev Company. Two companies, one operational area each:

COMPANY	CODE	FARM	CAPACITY	AREA
Apex Swine Genetics & Breeding Pvt Ltd	<code>APEXBREED</code>	Apex Nucleus Breeding Farm (Karnal)	165 head	Apex Nucleus Breeding & Gestation Unit
Highland Commercial Porkers & Processing Pvt Ltd	<code>HIGHLAND</code>	Highland Commercial Swine Facility (Hisar)	360 head	Highland Grow-Finish Commercial Complex

Five tenant users plus the platform admin. Password is `12345678` for all of them.

USER	EMAIL	TYPE	SEES
Rajesh Varma	<code>admin@apexagri.local</code>	<code>TENANT_ADMIN</code>	Both companies, all scopes
Dr. Arjun Sharma	<code>arjun.sharma@apexagri.local</code>	<code>COMPANY_ADMIN</code>	Apex only
Vikram Singh	<code>vikram.singh@highlandpork.local</code>	<code>COMPANY_ADMIN</code>	Highland only

USER	EMAIL	TYPE	SEES
Meera Nair	<code>supervisor@apexpork.local</code>	<code>OPERATIONAL_ADMIN</code>	Apex piggery area only
Suresh Rathi	<code>supervisor@highlandpork.local</code>	<code>OPERATIONAL_ADMIN</code>	Highland piggery area only
Platform admin	<code>admin@navfarm.local</code>	<code>SYSTEM_ADMIN</code>	<code>navfarm_master</code> — tenants, not farm data

Live data in `tenant_devco`: 5 batches, 39 animals, 307 batch transactions, 11 lifecycle stages, 4 schedulers with 50 parameter lines, 15 stage-transition logs, 17 journal headers, 6 posted cost variances.

2. The three scopes

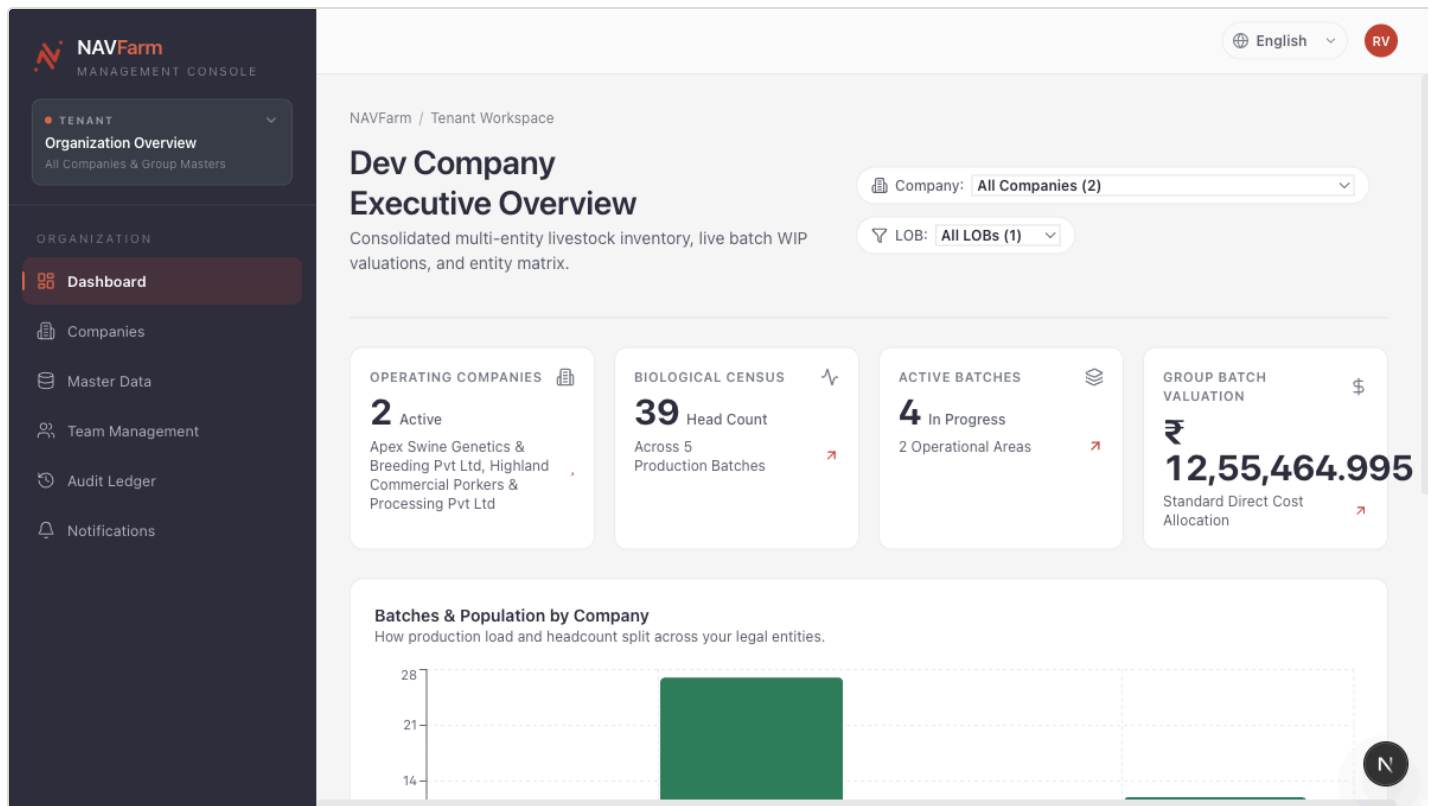
The scope is not a filter the user sets freely — it is derived from who they are, in

`apps/web/src/hooks/useAuth.ts`:

USER TYPE	SCOPE THEY GET	CAN THEY CHANGE IT?
<code>TENANT_ADMIN</code>	<code>TENANT</code> by default	Yes — down to <code>COMPANY</code> or <code>OPERATIONAL</code>
<code>COMPANY_ADMIN</code>	<code>COMPANY</code>	Only down to <code>OPERATIONAL</code> ; <code>TENANT</code> is refused
<code>OPERATIONAL_ADMIN</code> / <code>STANDARD_USER</code>	<code>OPERATIONAL</code>	No

`setActiveWorkspaceScope()` rewrites a `TENANT` request to `COMPANY` for anyone who is not a tenant admin, so the restriction holds even if the stored value is tampered with. The API does not trust any of this: every controller method carries `@RequirePermission(module, resource, action)` and the guard re-derives the user's rights from their role rows. The scope decides what the console *offers*; the guard decides what the server *allows*.

2.1 TENANT scope



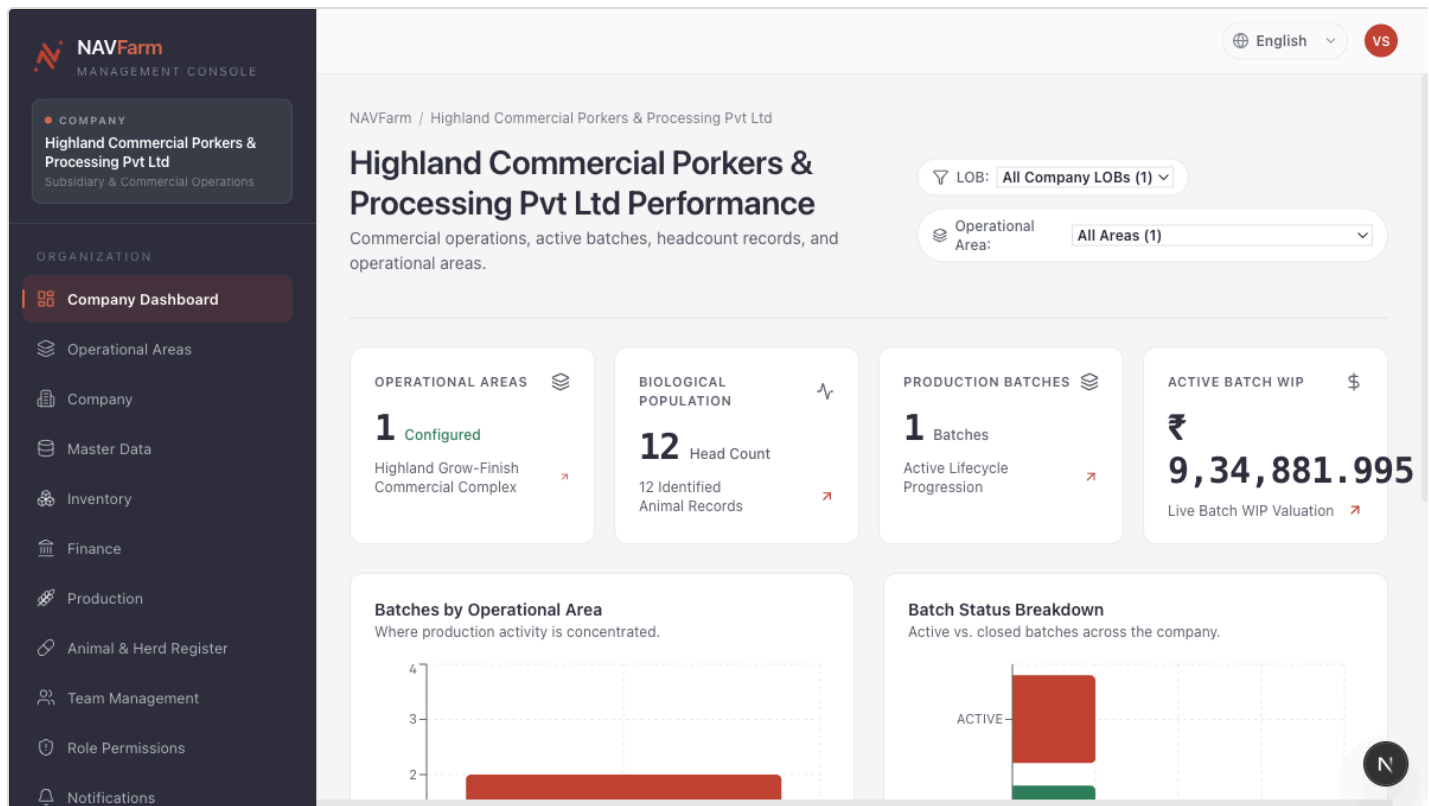
Tenant scope — executive overview

Signed in as Rajesh Varma. This is a consolidation view: operating companies, biological census across every company (39 head across 5 production batches), and live WIP valuation rolled up. The Company and LOB selectors at the top are the consolidation axis — "All Companies (2)" widens to both entities, picking one narrows without changing scope.

Why it exists. A tenant admin's questions are about the group, not the pen: which entity is carrying the most WIP, where the herd sits, which companies are configured. Nothing on this screen asks you to know a batch number.

What TENANT scope adds over COMPANY: the companies register, tenant-wide user and role administration, the audit trail, and notification channel configuration.

2.2 COMPANY scope



Company scope — Highland performance

Signed in as Vikram Singh (Highland). The header is the company, and the selectors are now LOB and Operational Area within it. The tiles answer a company controller's questions: how many areas are configured, headcount, active batches, WIP.

Why it exists. Finance and inventory are company-level facts. The journal, trial balance, P&L, balance sheet and IAS 41 bio-asset reconciliation all belong to a legal entity, and inventory balances are held per company warehouse. A supervisor should not be reading the general ledger; a tenant admin reading it needs to know *which* entity's ledger it is.

2.3 OPERATIONAL scope

NAV Farm
MANAGEMENT CONSOLE

COMPANY

Highland Commercial Porkers & Processing Pvt Ltd

Subsidiary & Commercial Operations

ORGANIZATION

- Company Dashboard
- Operational Areas
- Company
- Master Data
- Inventory
- Finance
- Production**
- Animal & Herd Register
- Team Management
- Role Permissions
- Notifications

English

VS

NAV Farm / Highland Commercial Porkers & Processing Pvt Ltd

Batch List

Piggery Operational Area — Lifecycle tracking, batch feed & health logs, and cost-allocated closing.

Batches

Batch lifecycle: input placement, daily consumption/mortality/output, and closing to finished inventory.

All statuses

Search...

+ New Batch

BATCH NO.	START DATE	METHOD	OPENING QTY	UNIT COST	STATUS	ACTIONS
PIG-BAT-2026-0101	2026-07-15	STANDARD	120.0000 HEAD	—	ACTIVE	
PIG-BAT-2026-0102	2026-03-01	STANDARD	100.0000 HEAD	6608.489796	CLOSED	

Showing 1–2 of 2

25 / page

Page 1 of 1

Operational scope — batch list

Signed in as a supervisor (or a company admin who stepped down into an area). The header shows **AREA** **PIGGERY** and everything is about the day's work: batches, stages, data entry, records, transfers, livestock, alerts, approvals, parameters.

Why it exists. This is the only scope where data is *created* in volume. It removes the choices a supervisor should never have to make — which company, which entity's ledger — and leaves the ones they must: which batch, which stage, which animals.

3. The batch lifecycle

3.1 Stages are configuration, not code

`stage_master` holds 11 piggery stages for this tenant's LOB. The sequence, durations and transition rules are all data:

SEQ	CODE	NAME	TYPICAL DAYS	MIN DAYS BEFORE MOVE	TRIGGER
1	QUARANTINE	Quarantine	30	14	AUTO_BY_DAY
2	GILT_GROWER	Gilt Grower Phase	77	56	MANUAL
3	FLUSH_SERVICE	Flush and Service / AI	10	5	EVENT_BASED
4	DRY_SOW_GESTATION	Dry Sow / Gestation	110	90	EVENT_BASED
5	FARROWING	Farrowing	3	1	EVENT_BASED

SEQ	CODE	NAME	TYPICAL DAYS	MIN DAYS BEFORE MOVE	TRIGGER
6	LACTATION	Lactation / Nursing	28	21	AUTO_BY_DAY
7	WEANING	Weaning	1	1	EVENT_BASED
8	BOAR_AI	Boar AI Station	—	30	MANUAL
9	CB_GROWER	CB Grower Phase	77	60	KPI_BASED
10	SLAUGHTER	Slaughter	1	1	EVENT_BASED
11	DISPOSED	Disposed / End of Life	—	0	MANUAL

`min_days_before_move` is enforced on transition, so a batch cannot be advanced out of gestation on day 3. `alt_next_stage_id` is what makes a *backward* move legal: `DRY_SOW_GESTATION`'s next stage is `FARROWING`, but its alternate next stage is `FLUSH_SERVICE` under the trigger condition `PREGNANCY_FAILED` — exactly the path used in section 5.

NAV Farm
MANAGEMENT CONSOLE

AREA - PIGGERY

Apex Nucleus Breeding & Gestation Unit

Apex Swine Genetics & Breeding Pvt Ltd
Farm Operations

ORGANIZATION

Piggery Dashboard

Batches

Batch List

Batch Stages

Batch Animals

Data Entry

Records & Logs

Batch Transfers

Schedulers

Livestock

Animal Register

Breeding & Insemination

Health & Mortality

NAV Farm / PIGGERY Area

Batch Stages

Piggery Operational Area — Lifecycle tracking, batch feed & health logs, and cost-allocated closing.

PRODUCTION BATCH: **PIG-BAT-2026-0001 — Yorkshire Parity 1-3...**

Active Lifecycle

Breed: Yorkshire · Type: Piggery Batch · Start Date: 2026-03-06 · Current Stage: Dry Sow / Gestation

→ Advance Stage + Add Stage

STAGE SEQUENCE & TIMELINE CONFIGURATION Total Stages: 11 (337 Standard Days)

SEQ.	STAGE CODE	STAGE NAME	STAGE TYPE	STANDARD DAYS	DAY FROM	DAY TO	STATUS	ACTIONS
1	QUARANTINE	Quarantine	PRE_PRODUCTIVE	30	1	30	Completed	Edit
2	GILT_GROWER	Gilt Grower Phase	PRE_PRODUCTIVE	77	31	107	Completed	Edit
3	FLUSH_SERVICE	Flush and Service / AI	PRE_PRODUCTIVE	10	108	117	Completed	Edit

Batch stages

The stepper reads `stage_master` and `batch_stage_log` — the completed/current/pending marks come from the batch's own transition history, not from the row's position in the table. A stage the batch never entered is pending even if it sits before the current one in sequence.

Why all 11 stages are shown even for a batch that will only pass through five. The page is the line-of-business lifecycle, not the batch's itinerary. Hiding the unreachable stages would make two batches of the same LOB render different lifecycles, and the operator would lose the context of where this batch sits in the whole production model.

3.2 Advancing a stage

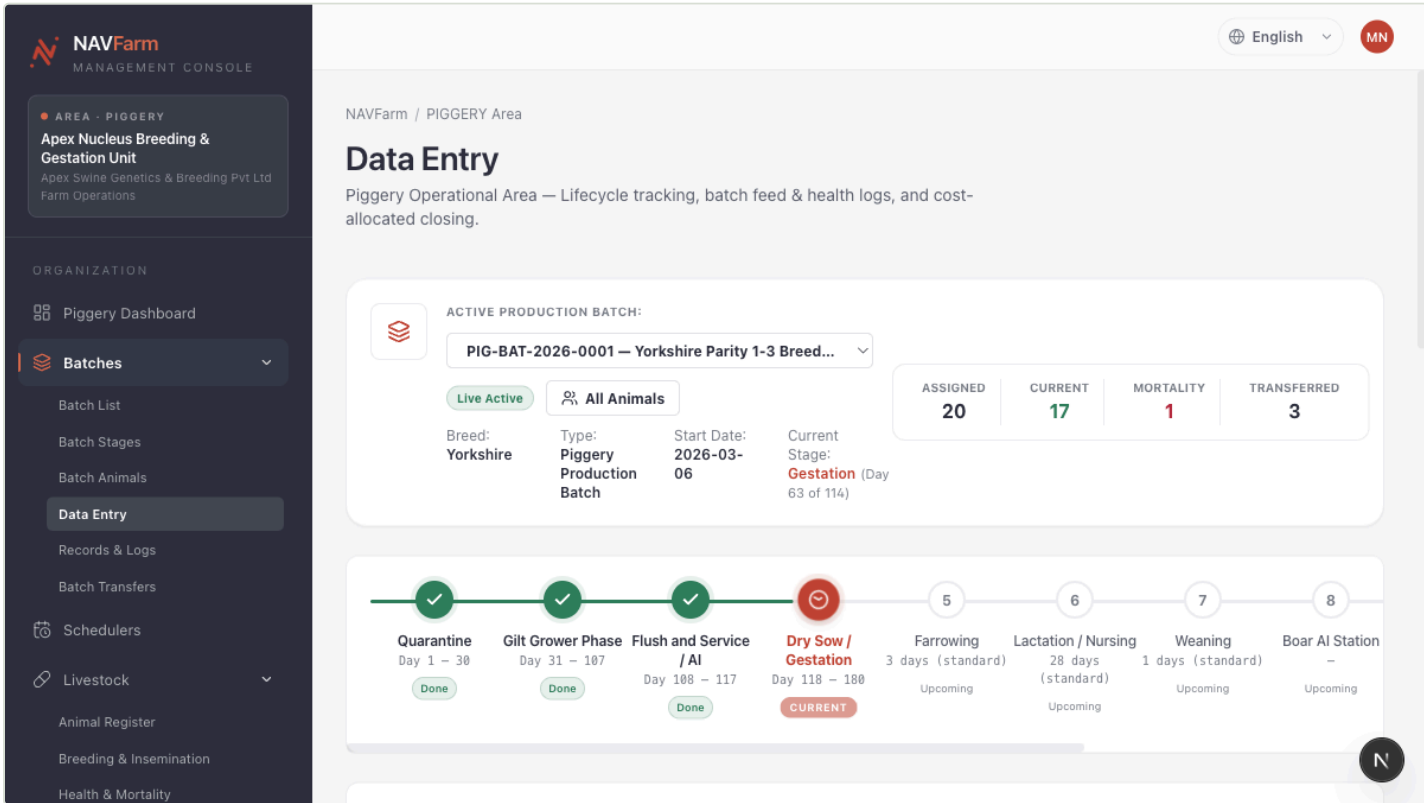
POST /batch/:id/transfer-stage moves the batch header (current_stage_code + stage_id), writes a batch_stage_log row, and cascades to the animals that are actually in the stage being left — an inArray update over the in-step animals only, so animals already held back in a different stage are not dragged forward. The endpoint validates to_stage_code against stage_master for the batch's LOB and rejects anything else with the valid list in the message:

```
'ST-05' is not a stage of this line of business. Valid stages: DRY_SOW_GESTATION, ...
```

That guard exists because a display-code array in the UI once posted ST-05 and corrupted batch_header.current_stage_code with a code no stage row matched, leaving stage_id null. The API now refuses it regardless of what any client sends.

4. Data entry and records

4.1 Data entry — the day's work



Operational data entry

Pick a batch, and the page resolves its real position: Current Stage: Gestation (Day 63 of 114) , with the lifecycle strip showing Quarantine (Day 1–30) and Gilt Grower (Day 31–107) already done. The day number is

computed from the *stage-entry* date in `batch_stage_log` , not from the batch start — otherwise every stage would report the same day.

The entry rows are driven by `scheduler_parameter_line` (section 6): a feed line for a gestation batch proposes the per-head gestation ration, not the nursery ration. The tiles above (`ASSIGNED 20` · `CURRENT 17` · `MORTALITY 1` · `TRANSFERRED 3`) reconcile the batch's animal population, so the operator can see at a glance that three head left and one died.

Entry can be made for the whole batch or for selected animals. A whole-batch entry writes one `batch_transaction` ; an animal-scoped entry writes one row per animal with `animal_id` set.

4.2 Records — what was entered

NAVFarm
MANAGEMENT CONSOLE

AREA · PIGGERY
Apex Nucleus Breeding & Gestation Unit
Apex Swine Genetics & Breeding Pvt Ltd
Farm Operations

ORGANIZATION

Piggery Dashboard

Batches

Batch List

Batch Stages

Batch Animals

Data Entry

Records & Logs

Batch Transfers

Schedulers

Livestock

Animal Register

Breeding & Insemination

Health & Mortality

NAVFarm / PIGGERY Area

Feed Management

Piggery Operational Area — Lifecycle tracking, batch feed & health logs, and cost-allocated closing.

BATCH

PIG-BAT-2026-0001 — Yorkshire Parity 1...

STAGE

DRY_SOW_GESTATION · DRY SOW GESTA...

DATE RANGE

01/07/2026 to 01/09/2026

ANIMAL

Whole batch

+ Log Consumption

Recalculate

Export

ANIMALS START

20

ANIMALS END

18

AVG AGE

45 d

DURATION

60 d

FEED CONSUMED

1,440.0 KG

MED COST

₹ 136

MORTALITY

2 (10.0%)

STAGE OUTPUT

18 Head

Feed Consumption (30)

Medicine & Clinical (1)

Labour & Manpower (1)

Overheads & Utilities (1)

Mortality Incidents (0)

N

Records and logs

The Records page is the read side of the same data, grouped by type — Feed Consumption, Medicine & Clinical, Labour & Manpower, Overheads & Utilities, Mortality Incidents, Weight & Growth, Notes & Logs, Stage & Pen Transfers — and filterable by the batch's real stage windows, e.g.:

QUARANTINE	2026-03-06 → 2026-04-04
GILT_GROWER	2026-04-05 → 2026-06-20
FLUSH_SERVICE	2026-06-21 → 2026-06-30
DRY_SOW_GESTATION	2026-07-01 → 2026-09-01

Those windows are derived from `batch_stage_log` , so they follow the batch's actual history.

Whole-batch vs per-animal records. An animal's record set is its own rows *plus an even share of the batch's unattributed rows* — implemented in `apportion-to-animal.ts`. Without that, an animal whose feed was always logged at batch level showed an empty record sheet, which was the original complaint. The share is `quantity / headCount`, which mirrors the write side: `CreateBatchTransactionDto` divides a row scoped to N animals evenly across them, so reading a whole-batch row back as an even share is the same arithmetic in reverse. Rows carrying another animal's `animal_id` are excluded, and a shared row is flagged `is_shared` so the UI can distinguish "this animal's own entry" from "this animal's share".

One limitation to know: the split uses the batch's current head count for every row, so it does not re-derive the head count as at each transaction's date. For a batch whose population changed mid-run, historical shares are approximate.

5. Partial stage moves — some animals are not ready

This is the case that shaped the transfer design: a gestation cohort of 20 sows, two of which fail the day-35 pregnancy scan. The other 18 must continue to farrowing; the two must go back to flush/service for re-breeding. They are the same cohort but can no longer share a stage.

NAVFarm
MANAGEMENT CONSOLE

AREA - PIGGERY
Apex Nucleus Breeding & Gestation Unit
Apex Swine Genetics & Breeding Pvt Ltd
Farm Operations

ORGANIZATION

Piggery Dashboard

Batches

Batch List

Batch Stages

Batch Animals

Data Entry

Records & Logs

Batch Transfers

Schedulers

Livestock

Animal Register

Breeding & Insemination

Health & Mortality

NAVFarm / PIGGERY Area

Batch Transfers

Piggery Operational Area — Lifecycle tracking, batch feed & health logs, and cost-allocated closing.

TRANSFERS
1

HEAD MOVED
2

VALUE MOVED
47,000

LATEST TRANSFER
2026-08-04

Move a whole batch, or selected animals, to another batch. Transferred animals stay operable — from the transfer date they simply belong to the destination batch.

TRANSFER NO.

DATE

MOVEMENT

SCOPE

HEAD

VALUE

BTR-2026-0002

2026-08-04

PIG-BAT-2026-0001 → PIG-BAT-2026-0003

Selected animals

2

47,000

New transfer

N

Batch transfers

The mechanism. A child batch with an explicit parent link:

1. `POST /batch-transfer/split/:batchId` creates a child batch with `parent_batch_id = <origin batch>`, resolves the requested `hold_stage_code` to a real `stage_id`, and delegates the animal movement to the ordinary `PARTIAL` transfer path — so a split is not a second, parallel implementation of "move animals".

2. The selected animals get `current_batch_id`, `current_stage_id` and `current_location_id` pointed at the child batch, its stage and its pen. They stay fully operable.
3. The origin batch advances normally. Its cascade only touches animals still in the stage being left, so the held-back animals are untouched.
4. `POST /batch-transfer/merge/:batchId` moves all live animals back to the parent and closes the child when the group rejoins.

In the demo data this is `BTR-2026-0002`: 2 head, ₹47,000 of book value, moved `PIG-BAT-2026-0001` → `PIG-BAT-2026-0003` on 2026-08-04, reason `PREGNANCY_FAILED`, status `POSTED`. `PIG-BAT-2026-0003` carries `parent_batch_id = PIG-BAT-2026-0001` and sits at `FLUSH_SERVICE` while its parent sits at `DRY_SOW_GESTATION`.

Why a child batch rather than a per-animal stage override. Three reasons:

- **Cost has to land somewhere.** The two held sows keep eating. A batch is the only object that accumulates cost, so a diverted group without its own batch would either have its feed charged to a cohort it is no longer part of, or not charged at all.
- **The scheduler follows the stage.** Both batches share `SCHED-PIG-GEST-114`. Because `loadActiveScheduleLines()` filters lines by the batch's current stage, the parent gets the gestation lines and the child gets the flush/service lines — one scheduler, two stages, two different daily plans, no duplicate configuration.
- **It is reversible and auditable.** The parent link means the console can show the child as a split group rather than an unrelated batch, and the merge returns the animals with a transfer document behind the move.

Data Entry and Records both follow the child: entry for `PIG-BAT-2026-0003` proposes flush parameters, and the two animals' records show their gestation history under the parent and their flush history under the child.

6. Schedulers

NAV Farm
MANAGEMENT CONSOLE

COMPANY

Highland Commercial Porks & Processing Pvt Ltd

Subsidiary & Commercial Operations

ORGANIZATION

- Company Dashboard
- Operational Areas
- Company
- Master Data
- Inventory
- Finance
- Production
- Animal & Herd Register
- Team Management
- Role Permissions
- Notifications

NAV Farm / Highland Commercial Porks & Processing Pvt Ltd

Scheduler

Piggery Operational Area — Lifecycle tracking, batch feed & health logs, and cost-allocated closing.

Schedulers

Period-based KPI monitoring plans, attached to a batch at creation.

All stages

All breeds

Any status

Used or not

New Scheduler

CODE	NAME	DURATION	STAGES COVERED	LINE
SCHED-PIG-FIN-90	Finisher Grow-Out & Slaughter Schedule	137 DAY(S)	CB GROWER, SLAUGHTER, QUARANT...	10
SCHED-PIG-GROW-60	Quarantine & 60-Day Weaner-to-Grower Schedule	60 DAY(S)	CB GROWER, QUARANTINE, + all-stag...	10

Showing 1-2 of 2

25 / page

Page 1 of 1

Schedulers

A scheduler is a period-based monitoring plan attached to a batch when the batch is created. One scheduler per batch — but the plan inside it is stage-aware.

`scheduler_parameter_line` carries an optional `stage_code`:

- `stage_code = 'DRY_SOW_GESTATION'` — the line applies only while the batch is in gestation.
- `stage_code IS NULL` — an all-stage line; it applies for the whole run.

`loadActiveScheduleLines()` selects the lines whose `stage_code` matches the batch's current stage, plus all the null-stage lines. That is what the list column means when it reads `CB GROWER, SLAUGHTER, QUARANTINE, + all-stage lines`.

So the answer to "is a scheduler for a batch, or per stage?" is **both**: it is attached per batch, and its lines are scoped per stage.

The demo tenant's four schedulers:

CODE	DURATION	ATTACHED	STATE
SCHED-PIG-GEST-114	231 days	2 batches (parent + split)	LOCKED
SCHED-PIG-FARR-28	280 days	1 batch	LOCKED
SCHED-PIG-GROW-60	60 days	1 batch	LOCKED
SCHED-PIG-FIN-90	137 days	1 batch	EDITABLE

Locking. A scheduler in use by an active batch is locked — editing the plan under a running batch would retroactively change what the batch was measured against. `FIN-90` is editable because its only batch (`PIG-BAT-2026-0102`) has closed; `syncSchedulerLock()` runs on close and releases it.

The list supports search and filtering by stage, and shows for each scheduler how many batches are attached, how many parameter lines it carries and which stages those lines target.

7. Costing

Two costing methods are in play, and they behave differently on purpose.

7.1 `BIO_ASSET` — the breeding herd

`PIG-BAT-2026-0001/0002/0003`. These animals are biological assets under IAS 41, not work-in-progress that will be converted to inventory. They carry a book value in `batch_bio_asset_state` and per-animal value in `animal_register` (`total_opening_asset_value`, `total_amortised`, `book_value`), with movements posted to `bio_asset_ledger`.

A `BIO_ASSET` batch cannot be closed through the batch-close action — the API says so explicitly:

BIO_ASSET batches don't close via this action — use the dispose endpoint to exit animals from the herd; the batch closes automatically once the herd is fully disposed.

That is the right model: a breeding cohort ends when its last animal leaves the herd, not on a date someone picks.

7.2 `STANDARD` — the grow-finish batches

`PIG-BAT-2026-0101/0102`. Cost accumulates as `batch_transaction` rows (`CONSUMPTION`, `OVERHEAD`) plus `batch_input_line` placements, and the batch closes into finished inventory at a standard rate, with the difference posted as variances.

WIP while open. `batch_header.total_cost` is only written at close, so an open batch has none. `GET /batch` therefore computes `wip_value` per batch — the sum of its `CONSUMPTION` and `OVERHEAD` amounts — falling back to the posted `total_cost` once closed. `MORTALITY` is excluded: it is expensed and relieved from WIP when it is recorded, and counting it again would double it into the surviving output's valuation.

Closing. `POST /batch/:id/close` with a closing quantity and output lines:

1. Accumulates actual cost = input lines + `CONSUMPTION` + `OVERHEAD`.
2. Values the output at `batch_standard.std_output_cost_per_unit` (not at a proportional split of actual cost) — which is what makes the variances a real reconciliation rather than a side report.
3. Computes PRICE, USAGE, OUTPUT and OVERHEAD variances against `batch_standard` and `batch_standard_consumption_line`.
4. **Refuses to close if it does not reconcile.** If actual cost $\neq$ standard output value + variances to within ₹0.01, nothing is written and the operator is told that a consumption has no matching standard line or a standard rate is unset.

5. Writes the inventory ledger entry, the GL journal, the output lines, and the variance rows.

PIG-BAT-2026-0102 was closed through this path during verification. 100 head in, 98 out, actual end 2026-07-15:

	AMOUNT
Actual accumulated cost	₹647,632.00
Standard output value (98 × ₹6,432.483948)	₹630,383.43
Total variance	₹17,248.57 (2.74%, unfavourable)

made up of six posted variance rows, each with its own GL journal:

TYPE	ITEM	STANDARD	ACTUAL	VARIANCE
PRICE	FEED-FINISHER	40.18 /unit	41.00 /unit	705.20
USAGE	FEED-FINISHER	834.20	860.00	1,036.64
PRICE	MED-IVERMECTIN	3.332 /unit	3.400 /unit	61.20
USAGE	MED-IVERMECTIN	873.00	900.00	89.96
OUTPUT	—	100 head	98 head	12,864.97
OVERHEAD	—	47,321.40	49,812.00	2,490.60

Unit cost landed at ₹6,608.49/head, written back to the batch header.

NAVFarm

MANAGEMENT CONSOLE

COMPANY

Highland Commercial Porkers & Processing Pvt Ltd

Subsidiary & Commercial Operations

ORGANIZATION

Company Dashboard

Operational Areas

Company

Master Data

Inventory

Finance

Production

Animal & Herd Register

Team Management

Role Permissions

Notifications

Costing & WIP Journals

Profit & Loss (P&L)

Balance Sheet

Trial Balance

IAS 41 Bio-Asset Statement

Batch Cost Variance

NAVFarm / Highland Commercial Porkers & Processing Pvt Ltd

Standard Cost Variance by Batch

General Ledger journals, WIP allocations, and statutory financial statements.

Batch Cost Variance

Actual cost against standard for every closed standard-costed batch, split by price, usage, output and overhead.

Search batch number.

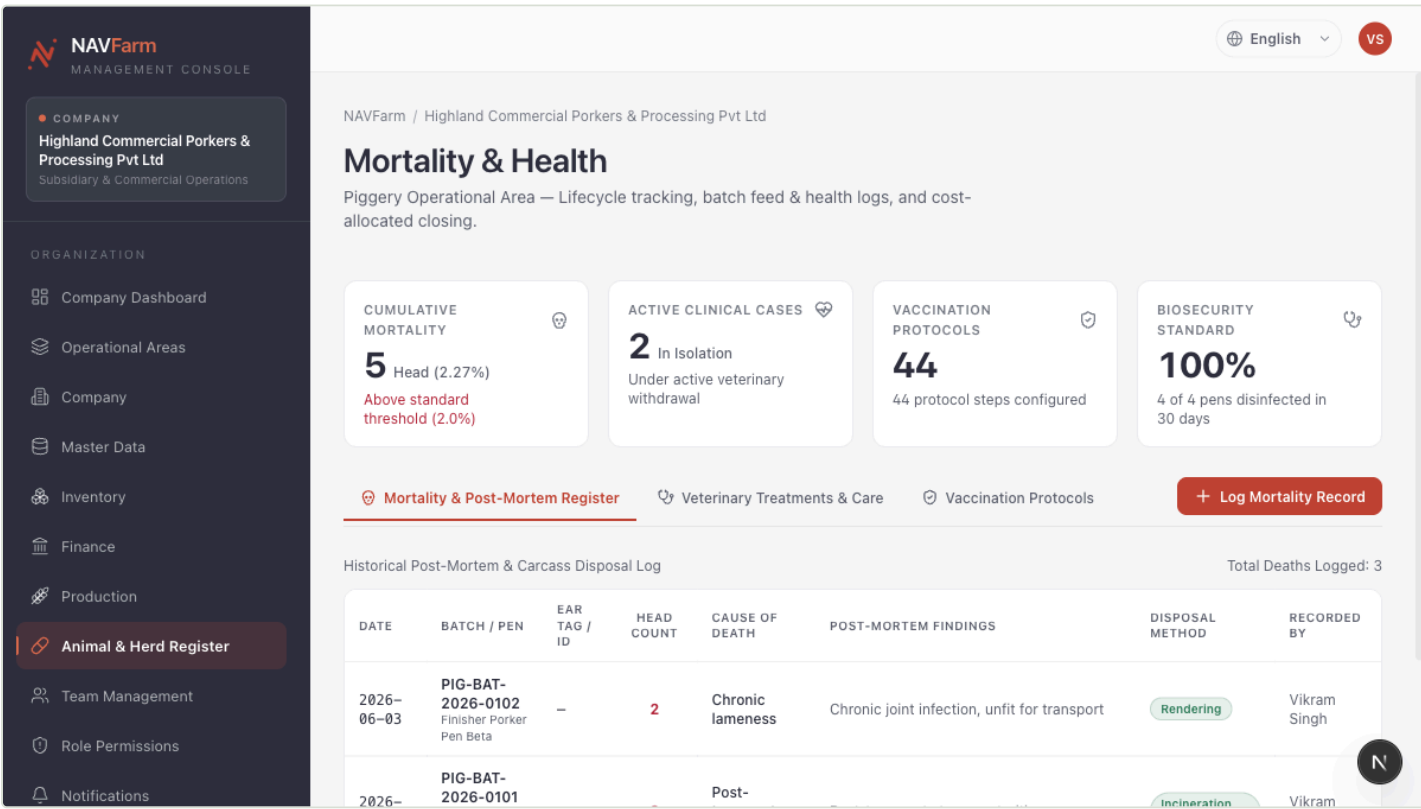
BATCH	METHOD	STANDARD COST	ACTUAL COST	PRICE	USAGE
PIG-BAT-2026-0102	STANDARD	₹6,30,383.43	₹6,47,632.00	₹766.40	₹1,126.61
Total across 1 batch(es)		₹6,30,383.43	₹6,47,632.00		

Batch cost variance

Finance → **Batch Cost Variance** shows this per batch: standard against actual, the four variance types side by side, and the total with its percentage. Unfavourable amounts are positive throughout the costing engine, so the sign carries the meaning and the colour follows it.

The report only has data after a batch closes. That is not a gap — variances are produced by `close()`, so a tenant with no closed standard batch legitimately has none, and the page says so rather than showing an error. The seed leaves `PIG-BAT-2026-0102` staged at slaughter weight and `ACTIVE`; closing it from the console is what fills the report.

8. Health, mortality and biosecurity



Mortality and health

Every figure on this page is computed:

- Cumulative mortality** — recorded deaths ÷ opening head across the batches in scope, compared against the 2.0% standard and coloured by the verdict. Highland is at 2.27%, so the card reads *above standard*, in danger colour.
- Active clinical cases** — treatments whose recorded withdrawal period has not yet elapsed, measured from the treatment date. A case stops being active on its own.
- Vaccination protocols** — the count of protocol steps configured on `breed_lifecycle_stage.vaccination_protocol` for this company's breeds.

- **Biosecurity** — the share of pens whose `last_disinfected_date` is within 30 days, shown as `4 of 4 pens disinfected in 30 days`.

Where clinical detail lives. A death or a treatment is one `batch_transaction` — the quantity and the cost — plus a 1:1 row of clinical narrative:

TABLE	COLUMNS
<code>batch_mortality_detail</code>	<code>location_id</code> (the pen), <code>cause_of_death</code> , <code>post_mortem_notes</code> , <code>disposal_method</code>
<code>batch_treatment_detail</code>	<code>diagnosis</code> , <code>route</code> , <code>withdrawal_days</code> , <code>veterinarian</code>

Both key on `transaction_id` with a unique constraint and `ON DELETE CASCADE`, so an event is still a single row of cost, nothing is duplicated, and deleting the transaction takes its narrative with it. `POST /batch/:id/transaction` accepts them as `mortality_detail` / `treatment_detail`, validated **before** anything is written — a `treatment_detail` on an `OVERHEAD` row is rejected with nothing persisted, and `route` must be one of the nine recognised administration routes.

`withdrawal_days` is the column that carries weight: it drives the active-case count and the slaughter withdrawal check, both of which were previously reading a number parsed out of prose.

This replaced a scheme where the detail was formatted into `remarks` and parsed back by the console, which made it unqueryable and easy to lose. The register still parses that old format as a fallback so rows recorded before these tables keep displaying.

9. Permissions

44 module/resource pairs are enforced by `@RequirePermission`. Role seeding (`default-role-seed.ts`) grants rights against exactly those pairs, derived from the `MASTER_DATA_RESOURCES`, `INVENTORY_RESOURCES` and `PRODUCTION_RESOURCES` lists — so a role cannot be granted a permission no controller checks, and a controller cannot check a permission no role can hold.

Authentication is JWT with refresh rotation; each refresh token carries a `jti`, which is what makes concurrent logins from several tabs safe. A session whose refresh is rejected is cleared and the browser is sent to `/login` rather than left on a signed-in page rendering zeroes.

10. How this was verified

Not from test output — from the running system.

Database. All 101 tables compared column-by-column against `schema.ts`: zero drift. 49/49 migrations applied to both tenant databases. Row counts cross-checked against what each screen displays.

API. All 43 list endpoints called with `companyId + limit + offset` — all 200. Three query DTOs were missing `limit / offset` and 400'd under the global `forbidNonWhitelisted` validation pipe; fixed. Finance reports all

return data.

Console. Every route walked in all three scopes, in the browser, signed in as the user who would actually use it. Screenshots in this document are from that walk.

End-to-end. `PIG-BAT-2026-0102` closed through the real API with real output lines, and the resulting variance rows, GL journals, inventory ledger entry and updated unit cost checked in the database and in the console.

Gates. API 215/215 tests, web 75/75 tests, API lint 0 errors, API typecheck 0 errors, web typecheck 0 errors, web lint 87 errors (at the accepted baseline, none new).

What this pass found and fixed

AREA	PROBLEM	FIX
Batch stages page	Rendered a hardcoded stage array with status assigned by row index	Fetches <code>/stage</code> , builds status from <code>batch_stage_log</code>
Stage transitions	A display code (<code>ST-05</code>) could be written into <code>current_stage_code</code> , leaving <code>stage_id</code> null	API validates against <code>stage_master</code> and rejects with the valid list
Data entry	Barn climate was a literal string	Reads <code>OBSERVATION</code> transactions
WIP tile	Structurally zero for every open batch	<code>GET /batch</code> computes <code>wip_value</code>
Batch list	Breed and stage rendered as <code>-</code>	List returns <code>breed_name</code> and <code>stage_name</code>
Standard costing	<code>batch_standard</code> was seeded with five columns that do not exist — every standard was <code>NULL</code>	Baselines derived from actuals so a close reconciles exactly
Closed batch	<code>CLOSED</code> was written straight to the row, skipping <code>close()</code> — no output valuation, no cost, no variances	Batch is staged <code>ACTIVE</code> ; closing it in the app posts everything
Variance report	Summed <code>std_value</code> / <code>actual_value</code> across variance types where they hold a rate, a quantity and an amount	Cost columns derived from the batch's own accumulated cost
Health page	Mortality %, vaccination coverage and biosecurity grade were literals; pen, post-mortem, disposal, route, vet and withdrawal were invented per row	All computed or parsed from what was recorded
Mortality dialog	Post-mortem notes and disposal method were collected then discarded	Round-tripped through <code>remarks</code>
Farm and shed capacity	<code>0</code> everywhere; <code>total_area</code> / <code>capacity_uom</code> were silently dropped by Drizzle (no such columns)	Real capacities on real columns, backfilled on re-seed
Split batch	The hold group had no <code>parent_batch_id</code>	Linked, and repaired on re-seed
Dead session	Left the user on a dashboard of zeroes with 14 console errors	Session cleared and redirected to <code>/login</code>
Roles	Seeded permissions for module/resource pairs no controller guards	Rebuilt from the enforced pairs
Auth	Concurrent logins returned <code>201 500 500</code>	Refresh tokens carry a <code>kti</code>

AREA	PROBLEM	FIX
Clinical records	Post-mortem and prescription detail were formatted into <code>remarks</code> and parsed back	<code>batch_mortality_detail</code> / <code>batch_treatment_detail</code> , validated before any write
Variance report	API-only — no console page	Finance → Batch Cost Variance, in all 8 locales
API typecheck	48 errors, including 20 dead <code>success: true</code> literals overwritten by a spread and a <code>batch_code</code> field that does not exist on <code>batch_header</code>	0 errors
Animal lookup	<code>batch_code</code> was always <code>undefined</code> — the column is <code>batch_no</code>	Returns <code>batch_no</code>
<code>findAll</code> return types	Early <code>return rows</code> widened the type so callers could not read <code>wip_value</code> / <code>line_count</code>	Always returns the enriched shape

Known gaps

- **Only Piggery and Dairy have purpose-built screens.** `lob_master` ships 16 lines of business across 6 natures of business, and `LOB_HAS_DEDICATED_PANELS` marks two of them as having dedicated panels; the other fourteen fall back to the generic batch screens. Those work — everything hangs off a batch — but a poultry or aquaculture operator gets no species-specific vocabulary or KPIs.
- **Mortality and treatment must belong to a batch.** The clinical detail tables extend `batch_transaction`, so a death or treatment for an animal that is not currently in a batch has nowhere to go. Every animal in the demo data is in a batch, so this is not yet reachable.
- **The mortality dialog has no pen picker.** It takes free text and files it as the transaction's remark; `batch_mortality_detail.location_id` falls back to the batch's current location. Wiring the field to a real pen selector is a small change.
- **Web lint baseline is 87–88 errors**, mostly `no-empty` on `catch {}` and `react-hooks/exhaustive-deps`. The gate is "no new errors", not zero.

11. Rebuilding the demo environment

```
pnpm nx run api:db-seed-demo --args="--fresh" # drops and rebuilds the three databases
pnpm nx serve api                             # :2877
pnpm nx serve web                             # :3002
```

`--fresh` drops `navfarm_master`, `tenant_system` and `tenant_devco` by name — no other database on the MySQL instance is touched. The seed is idempotent: running it without `--fresh` repairs and backfills in place rather than duplicating.

To see the full standard-costing flow, sign in as `vikram.singh@highlandpork.local`, open Batches, and close `PIG-BAT-2026-0102` at 98 head with an actual end date of 2026-07-15.